MONASH University

Monash eSolutions

eSolutions Welcomes FIT3143 Students to the Cluster As A Service (CAAS) Platform



GROUP OF EIGHT AUSTRALIA

Agenda

1. HPC overview
    a. Basic architecture: how-to access & use CAAS
    b. Tools required for accessing
    c. account information, password, and signing into CAAS
2. Basic HPC (Linux) Command & Usage
    a. navigation, file management, directory management
    b. $HOME and Project Folders on CAAS
    c. running the text editor
    d. managing data transfers
3. How to use a compute cluster
    a. basic concepts
    b. SLURM commands
    c. submitting your first job, checking results
4. Further reading

Contributors

1. CAAS Platform Setup by eSolutions
    a. Contact: Justin Robins (justin.robins@monash.edu), Zoltan Deak Jnr (zoltan.deak@monash.edu)
2. Documentation
    a. Original owner: Philip Chan (mcc-help@monash.edu)
    b. Revisions made by:
        i. Simon Forsayeth (simon.forsayeth@monash.edu)
        ii. Terrence Mak (terrence.mak@monash.edu)
        iii. ABM Russel (abm.russel@monash.edu)
        iv. Vishnu Monn (vishnu.monn@monash.edu)
        v. Jessie Leong Shu Min (leong.shumin@monash.edu)

Note: The content of this document will be revised during the semester. These revisions are based on updates or changes made to the CAAS platform.

Introduction to High-Performance Computing

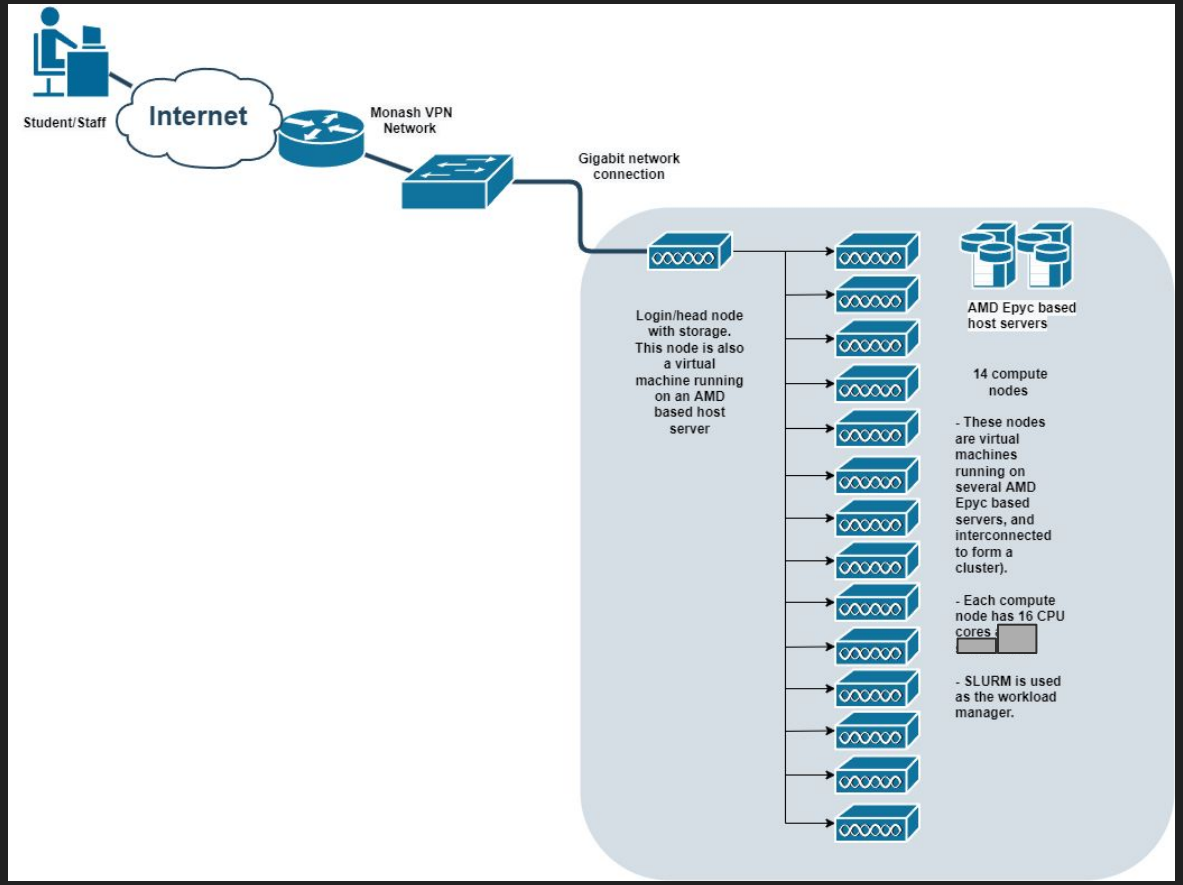
- High-performance computing is about employing the best and cost-effective technologies, from processors, storage and networks, to provide aggregated computational capabilities beyond what is typically available to the end-user
- HPC versus HTC
  - high-performance -- running a single program as quickly as possible
  - high-throughput -- running many independent programs as quickly as possible
- HPC/HTC is predominantly achieved by one or more compute clusters
- HPC/HTC are enabling technologies for larger experiments, more complex data analyses, achieving higher accuracy in computational models

Introducing CAAS

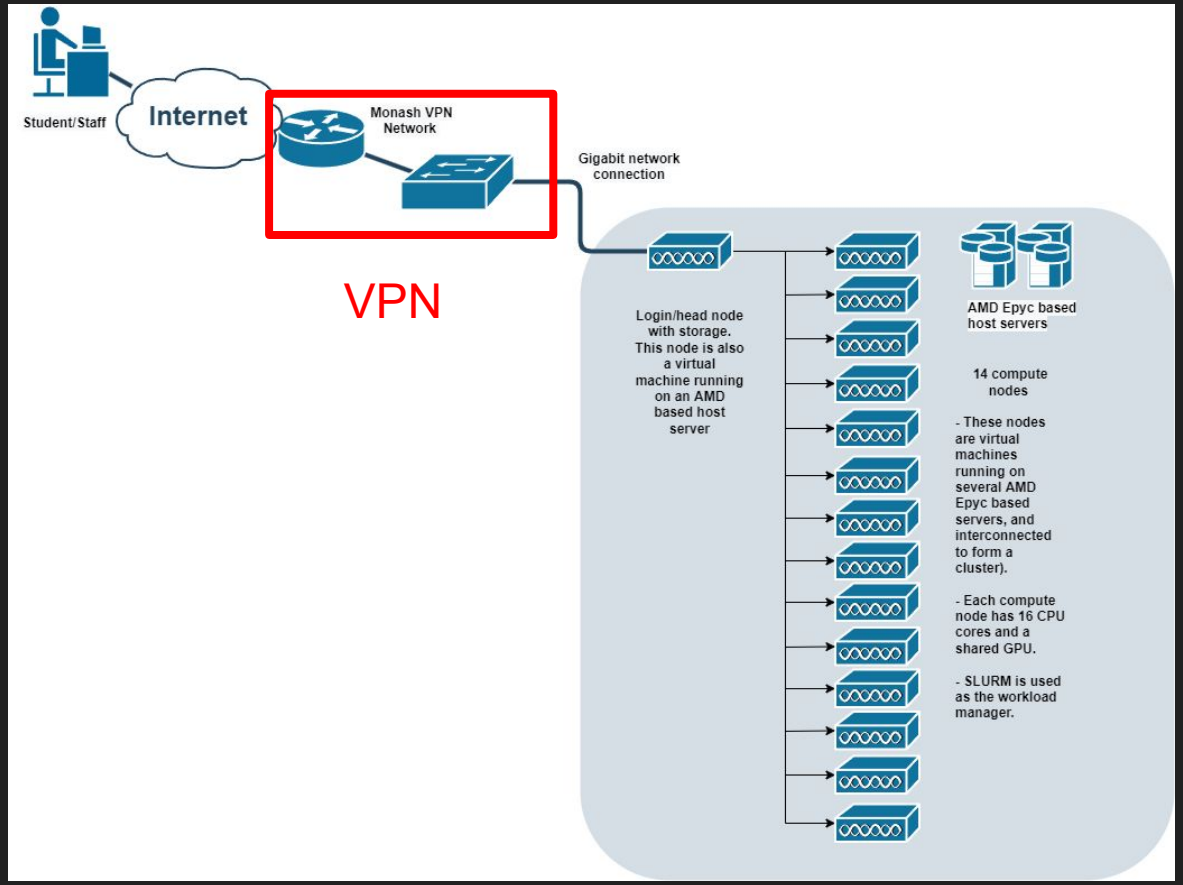
Cluster As A Service (CAAS)

- university cluster for education focused high-performance computing
- open for access to Monash students (based on specific units)
- made up of over 14 compute nodes connected via Gigabit network
- each compute node has 16 CPU cores and a shared GPU.
- runs on top of the Monash network
- requires Monash VPN to access the platform for students.
- **Note:** Nodes may be shared with research clusters

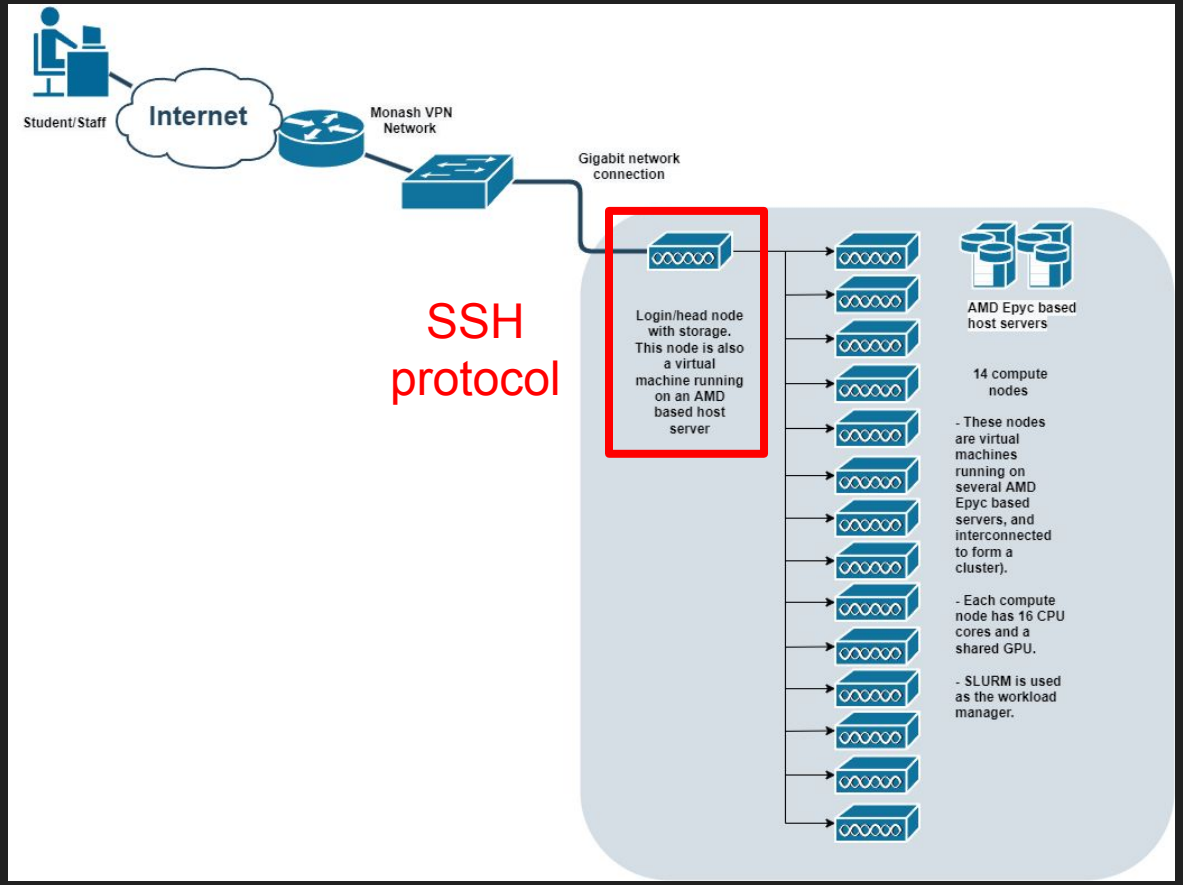
CAAS basic architecture



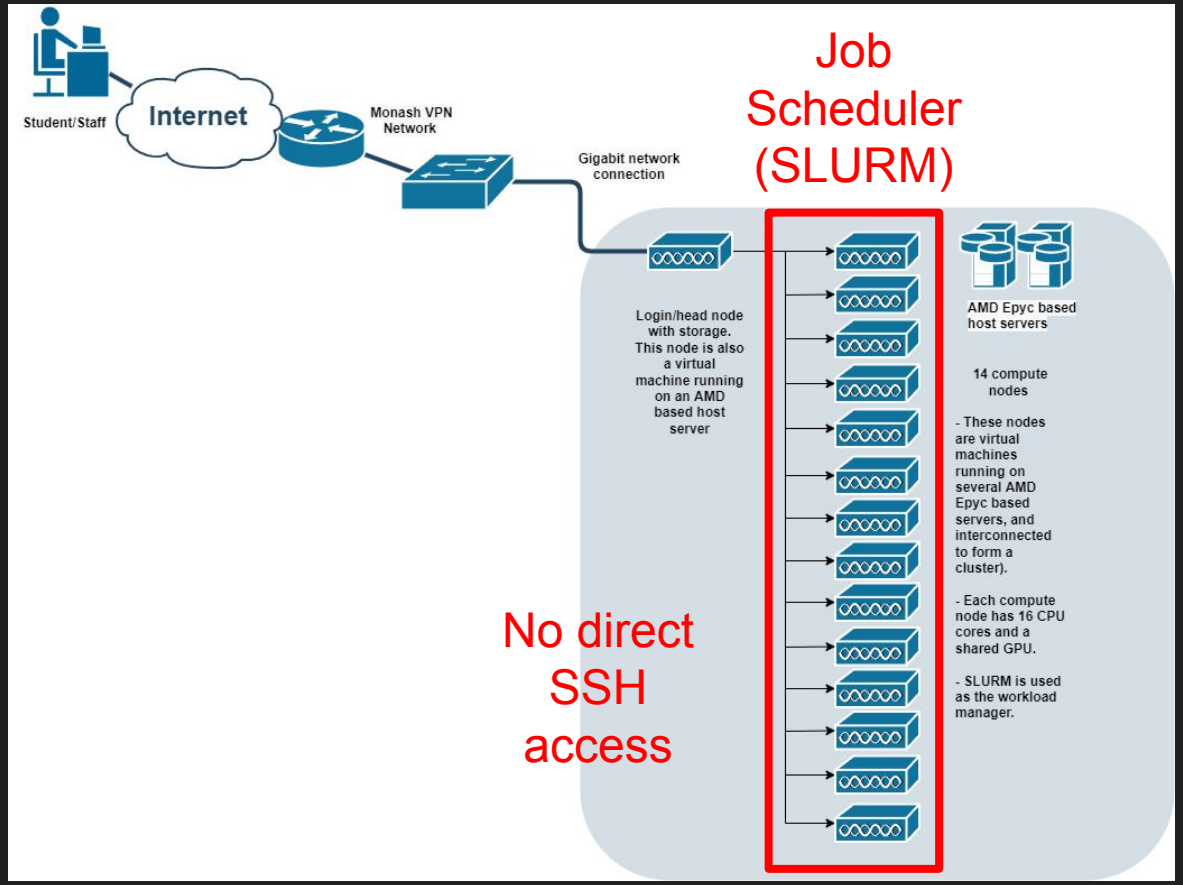
CAAS basic architecture



CAAS basic architecture



CAAS basic architecture



CAAS vs Monash HPC (M3 or MonARCH)

M3 - High Performance Computing (HPC) facility designed specifically to process complex data for **research**. It is based on a cluster of CPU and GPU compute nodes.

- Total number of cores: 5673
- Total number of GPU cores: 1,752,576 CUDA cores
- Total memory: 63.2 TBytes
- Storage capacity: 5.6 PBytes Lustre parallel file system
- Network: Mellanox Infiniband (Up to 200 Gbps)
- Utilises **SLURM** scheduler
- Administration by eResearch

CAAS - High Performance Computing (HPC) system designed specifically for **education purposes**. It is also based on a cluster of CPU and GPU compute nodes. The aim of CAAS is provide a real-world like experience in using a HPC.

- Total number of cores: 256+ (AMD Epyc server CPUs)
- Total memory: 256 GBytes +
- Storage capacity: 5 TBytes +
- Network: Gigabit ethernet
- **Utilises SLURM scheduler**
- Administration by eSolutions

FIT3143 Students/Staff to use CAAS

M3: https://docs.erc.monash.edu/M3/

What you need to access CAAS

Monash VPN

- You must be connected to Monash VPN first (sometimes even if you're in campus)!
- VPN Gateway: vpn.gp.monash.edu
- Instructions:
  - https://www.monash.edu/esolutions/network/vpn
  - Latest client will needed to be installed.
  - Please choose according to your operating systems

What you need to access CAAS

SSH Client

- On Linux, Mac, or WSL (and many other) environments, the commands **`ssh`** and **`scp`** (for file copying) are likely pre-installed (i.e., you already have a client)

- If you don't like command line clients, you can always install other clients yourself. For example, FileZilla is available on these platforms.

- Famous client (Windows)
  - Putty (the famous **`ssh`** client)
    - https://www.putty.org/
  - WinSCP (secure file copying client)
    - https://winscp.net/download/WinSCP-6.3.4-Setup.exe/download
  - `FileZilla` (secure file copying client)
    - https://filezilla-project.org/download.php?show_all=1

Logging into CAAS

FIT3143 staff and students are registered on CAAS; and your user account is based on your Monash authcate credentials.
Your Monash authcate username is the word before the @ in your email address, e.g.:

`abcd0001@student.monash.edu`

The Remote Host address (using terminal):

ssh **`student-caas-headnode.rep.monash.edu (with IP 172.23.169.56)`**

When prompted for a username:

- Use your Monash authcate username
- Use your **MyMonash** password.

Reminder: You need to connect to Monash VPN before carrying out the step above. Refer to https://www.monash.edu/esolutions/network/vpn for more information about connecting to Monash VPN.

On successful SSH access into CAAS

You will see a "`Monash eSolutions - HPC-as-a-Service`" message.

Below the message, you should see a command prompt, e.g.,

`[yourAuthcate@student-caas-headnode ~]$`

awaiting your textual commands.

```
Monash eSolutions - HPC-as-a-Service
------------------------------------
Based on Red Hat Enterprise Linux 8


This system is hosted on the Monash Private Cloud. Access is
restricted to Monash University authorised user
s. The use of this system is subject to \
Monash University's IT Acceptable Use Policy and is subject to
regular monitoring. Unauthorised or improper
use of this system may result in adminis\
trative disciplinary action and/or civil and criminal penalties.
Use of this system constitutes consent to security monitoring.

----------------------------------------------------------------
--------------
Use the following commands to adjust your environment:

'module avail'            - show available modules
'module add <module>'     - adds a module to your environment
for this session
'module initadd <module>' - configure module to be loaded at
every login
----------------------------------------------------------------
--------------
```

Agenda

1. HPC overview
    a. Basic architecture: how-to access & use CAAS
    b. Tools required for accessing
    c. account information, password, and signing into CAAS
2. Basic HPC (Linux) Command & Usage
    a. navigation, file management, directory management
    b. $HOME and Project Folders on CAAS
    c. running the text editor
    d. managing data transfers
3. How to use a compute cluster
    a. basic concepts
    b. SLURM commands
    c. submitting your first job, checking results
4. Further reading

First Commands on CAAS

You are now within your “home folder”, we call **$HOME**, also called **~**

First commands:

`pwd`	-	displays the working directory (where you are)
`echo $HOME`	-	try this command and compare its output with above
`date`	-	display the system date and time
`cal 2025`	-	show the calendar for 2025
`clear`	-	clears the terminal window
`sinfo`	-	shows a summary of active nodes on the platform

Important: Do not attempt to change your authcate password via this platform. If you need to change password, please contact Monash Connect for assistance.

Files, Directories and Paths

In Unix/Linux, everything is a file; even a directory is just a special file.

Each file on the system is identifiable by its absolute path.

A path is a string that specifies the name & location of a file (or directory); it consists of characters separated by zero or more / (“forward slash” character)

Examples of absolute paths:

```
/home/YourAuthcate
/usr/local/dos2unix/7.4.0/bin/dos2unix
```

are there relative paths?

Further reading: https://en.wikipedia.org/wiki/Path_(computing)

Files, Directories and Paths (relative paths)

A relative path is one that does not have a leading / (“forward slash” character)

Each file on the system is identifiable by its absolute path. Any absolute path can also be specified as a relative path.

Examples of relative paths:

```
python          ?≡?  ./python
bin/dos2unix     ≡   ./bin/dos2unix
```

two special relative paths:

- `.` - this directory
- `..` - the parent directory

Use of relative paths in your program **can break your access to data** if the program is run from another folder on the system.

Your personal and project spaces on CAAS

`${HOME}` - your personal $HOME directory (5 GB quota)
good for source codes, configuration files, etc.
not good for HPC with lots of data.

Basic Navigation and Directory Management

`ls` - list the file names in current directory

`ll` - alias for ls -l . The option -l tells the command to use a long list format.

`mkdir` *path* - create an empty directory

`rmdir` *path* - removes an empty directory

`cd` *path* - change into the given path/directory

`cd ..` - change into the enclosing directory

`cd` - change into your `$HOME` directory

`tree` - visualise the directory structure

Or try this sequence:

```
cd; mkdir A; cd A; mkdir a b c; cd; tree
```

Something to try:

```
cd
mkdir 2024
ls
cd 2024
mkdir jan feb mar
ls
cd jan
mkdir wk1
cd ../mar
mkdir wk1
cd ../feb
mkdir wk1 wk2
cd
tree
```

Navigating your way within CAAS

Key commands:

`cat` *path*	-	displays the contents of *path* (try this on a folder?)
		be careful running this on a binary file
`cp` *src dest*	-	makes a copy of *src* into *dest* (both are paths)
`mv` *src dest*	-	moves or renames files (or folders)
`rm` *path*	-	permanently deletes the given file
`cd`	-	go back `$HOME`
`cd` *path*	-	change into the given path / folder

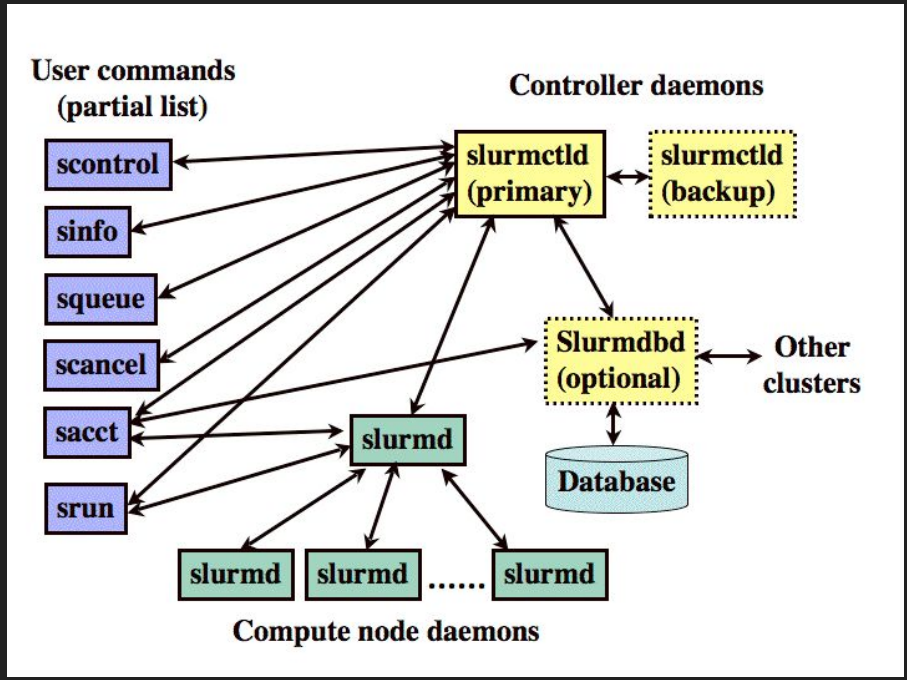
Agenda

1. HPC overview
    a. Basic architecture: how-to access & use CAAS
    b. Tools required for accessing
    c. account information, password, and signing into CAAS
2. Basic HPC (Linux) Command & Usage
    a. navigation, file management, directory management
    b. $HOME and Project Folders on CAAS
    c. running the text editor
    d. managing data transfers
3. How to use a compute cluster
    a. basic concepts
    b. SLURM commands
    c. submitting your first job, checking results
4. Further reading

SLURM Scheduler

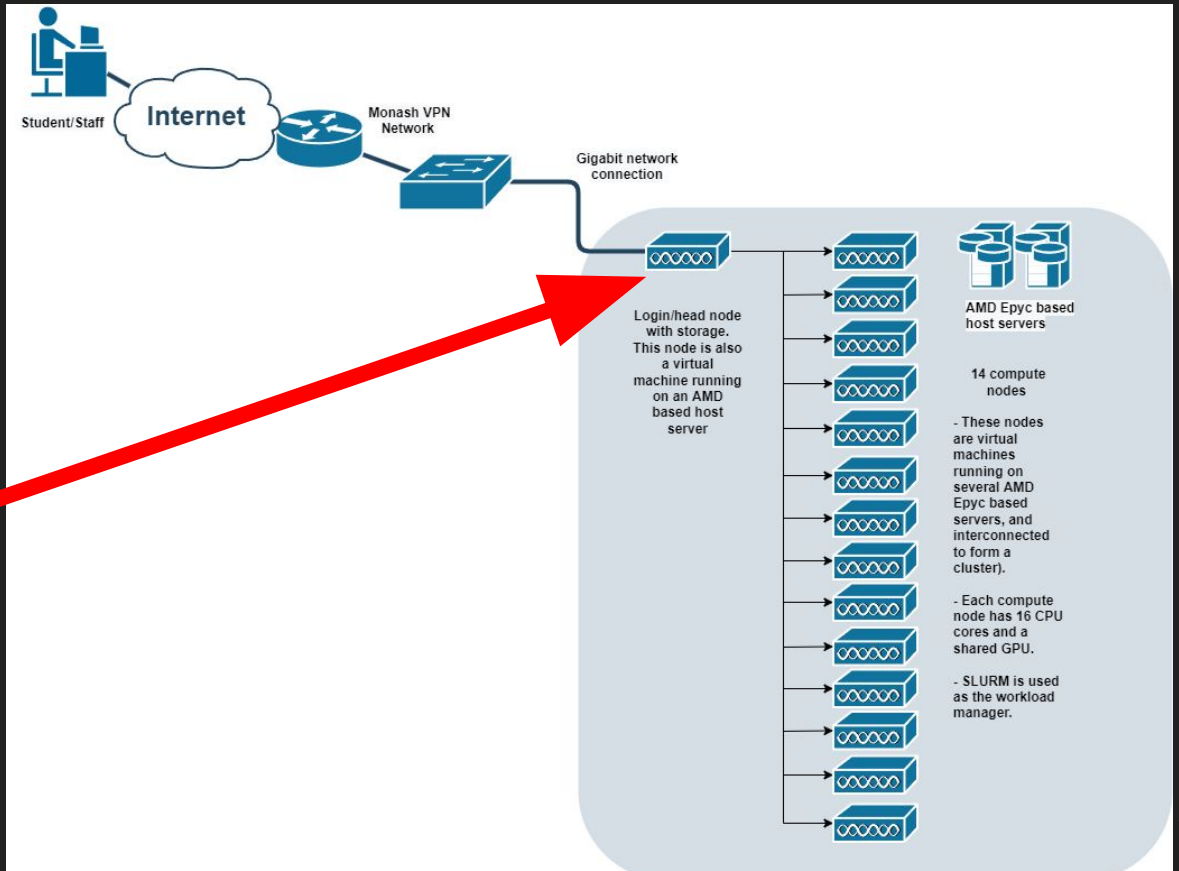
CAAS uses SLURM as its resource manager and job scheduler:

- fair share scheduling policy
- no fixed CPU allocation, jobs are prioritised based on history of usage

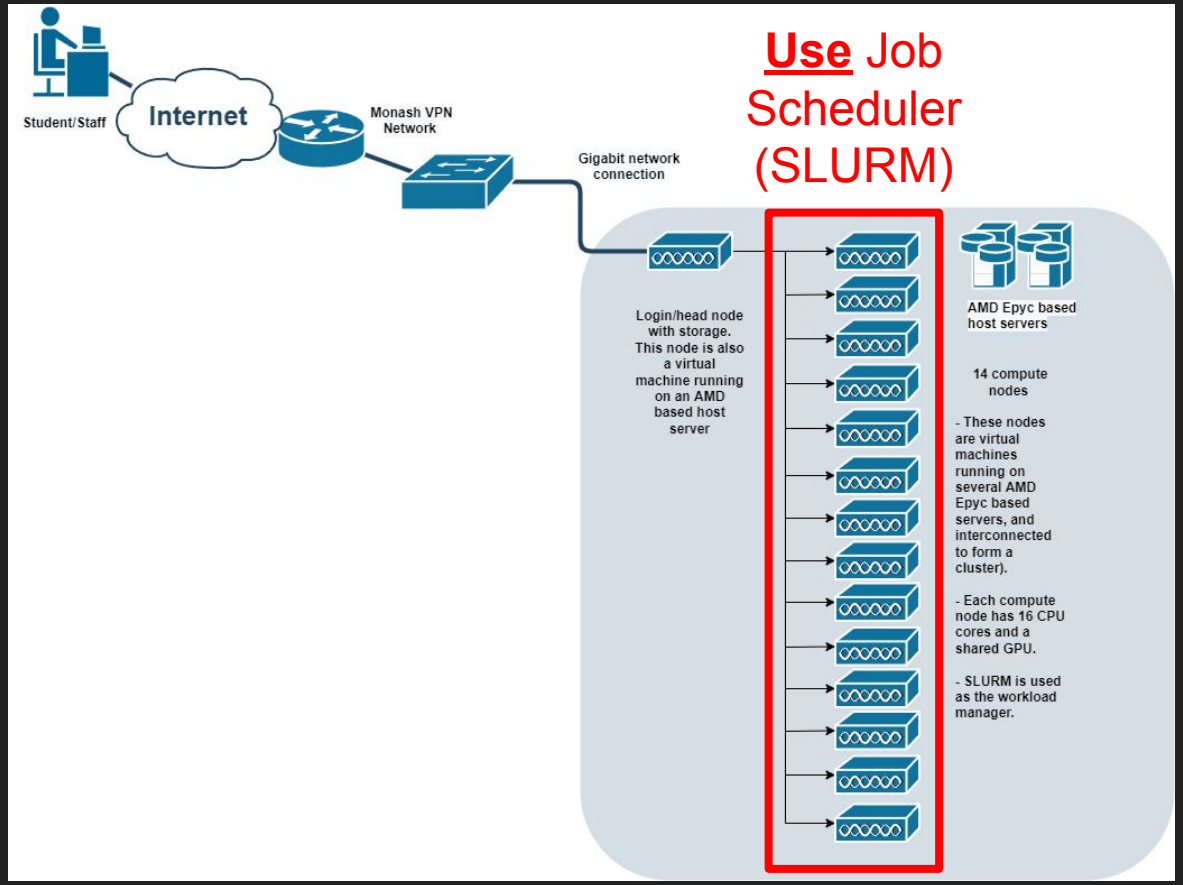


Source: https://slurm.schedmd.com/overview.html

Do not run your program directly in the Head Node!!!



CAAS basic architecture



Interacting with the SLURM Job Scheduler

`sbatch jobscript.sh`	- to submit a job for batch execution; returns the unique JobID if successful
`squeue -u ${USER}`	- to display the status of my jobs on the queue
`scontrol show job` *{jobid}*	- to check the status of a job and other details
`sacct -j` *{jobid}*	- to check the accounting information of a given job
`scancel` *{jobid}*	- to delete a job that you own given its unique JobID
`scancel -u ${USER}`	- to delete all your jobs on the cluster
`sinfo`	- to view partition and node information
`sinteractive`	- to start an interactive job. Takes standard SBATCH opts
`showq`	- to display a summary of active and queued jobs

To submit a job

1. Prepare your code and job file on your local computer. You may refer to the sample CAAS_DEMO.zip file in Moodle. Configure the job file.

2. Transfer the job and source files to CAAS using scp or filezilla:

   ```
   scp local_path caas_path
   ```

3. Unzip or check to ensure files have been transferred to CAAS.

4. Submit the job file:

   ```
   sbatch yourJob.job
   squeue
   ```

5. Check for output:

   ```
   ll
   tree
   cat slurm-*.out
   ```

MPI job script template
(MPI program with 8 MPI processes)

```
#!/bin/bash
#SBATCH --job-name=np_1x8_job      ### name your job
#SBATCH --time=00:10:00            ### hh:mm:ss or dd-hh:mm:ss
#SBATCH --mem=16G                  ### set this to 8 ntasks x 2 GB
#SBATCH --ntasks=8                 ### launch one mpi process
#SBATCH --cpus-per-task=1          ### single-threaded processes
#SBATCH --ntasks-per-node=8        ### set this to ntasks if ntasks <= 16

#SBATCH --partition=defq

module load openmpi/4.1.5-gcc-11.2.0-ux65npg
srun path-to-mpi-executable
```

10 minutes max per job

1 cpu max per task

Default partition

OpenMPI 4.1.5

one running max per user

MPI job script template
(MPI program with 8 MPI processes on 2 nodes)

```
#!/bin/bash
#SBATCH --job-name=np_2x4_job      ### name your job
#SBATCH --time=00:10:00            ### hh:mm:ss or dd-hh:mm:ss
#SBATCH --mem=16G                  ### memory setting is max @ 2 GB per core
#SBATCH --ntasks=8                 ### launch one mpi process
#SBATCH --cpus-per-task=1          ### single-threaded processes
#SBATCH --ntasks-per-node=4        ### four per node

#SBATCH --partition=defq

module load openmpi/4.1.5-gcc-11.2.0-ux65npg
srun path-to-mpi-executable
```

32 tasks is the maximum per job

10 minutes max per job

8 tasks for this job

1 cpu max per task

4 tasks per node, two nodes

Default partition

one running max per user

MPI job script template
(MPI program with 32 MPI processes on 2 nodes)

32 tasks is the maximum per job

```
#!/bin/bash
#SBATCH --job-name=np_32_job      ### name your job
#SBATCH --time=00:10:00           ### hh:mm:ss or dd-hh:mm:ss
#SBATCH --mem=16G                 ### memory setting is max @ 500 MP per process/core
#SBATCH --ntasks=32               ### equivalent to -np 32
#SBATCH --cpus-per-task=1         ### single-threaded processes
#SBATCH --ntasks-per-node=16      ### this is the max as each node has 16 cores only

#SBATCH --partition=defq

module load openmpi/4.1.5-gcc-11.2.0-ux65npg
srun path-to-mpi-executable
```

10 minutes max per job

32 tasks max per job

1 cpu max per task

Default partition

one running max per user

MPI + OpenMP job: 4 MPI processes, 4 threads per processes

```
#!/bin/bash
#SBATCH --job-name=mpi_omp_4x4_job     ### name your job
#SBATCH --time=00:10:00                ### hh:mm:ss or dd-hh:mm:ss
#SBATCH --mem=16G                      ### memory setting is max @ 4 GB per process
#SBATCH --ntasks=4                     ### mpi # of processes
#SBATCH --cpus-per-task=4              ### multi-threaded processes
#SBATCH --ntasks-per-node=2            ### two processes per node

#SBATCH --partition=defq

module load openmpi/4.1.5-gcc-11.2.0-ux65npg
export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
srun path-to-mpi-executable
```

16 cpu cores allocated to this job, 8 cores per node

10 minutes max per job

4 tasks for this job

4 cpu per mt process

2 tasks per node, two nodes

Default partition

one running max per user

OpenMP job: 1 process with 8 threads

```
#!/bin/bash
#SBATCH --job-name=mpi_omp_job      ### name your job
#SBATCH --time=00:10:00             ### hh:mm:ss or dd-hh:mm:ss
#SBATCH --mem=16G                   ### memory setting is max @ 2 GB per core
#SBATCH --ntasks=1                  ### launch one process
#SBATCH --cpus-per-task=8           ### multi-threaded processes

#SBATCH --partition=defq  ### use the default partition

export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
./path-to-executable
```

8 cpu cores allocated to this job on one compute node

10 minutes max per job

1 task (process) for this job

8 threads

one running max per user

pThread job script template *(8 threads on a single node)*

8 cpu cores allocated to this job on one compute node

```
#!/bin/bash
#SBATCH --job-name=mpi_omp_job      ### name your job
#SBATCH --time=00:10:00             ### hh:mm:ss or dd-hh:mm:ss
#SBATCH --mem=16G                   ### memory setting is max @ 2 GB per core
#SBATCH --ntasks=1                  ### launch one process
#SBATCH --cpus-per-task=8           ### multi-threaded processes

#SBATCH --partition=defq

./path-to-executable
```

10 minutes max per job

1 task (process) for this job

8 threads

one running max per user

Some thinking/considerations …

1. What happens if you require/specify more resources (CPU core/memory) in the SLURM submission script than what a machine (in the cluster) has?
2. What happens if your C program spawns 16 processes (or threads), but you only specify "`--ntasks=1`" and "`--cpus-per-task=1`"?
3. What happens if your C program uses 4GBs of memory (e.g., through calloc), but you only specify "`--mem=1G`"?
4. What happens if you accidentally runs your program in the head node (instead of using SLURM)? How to fix it?
5. What happens if your program runs in 1hr, but you specify "`--time=00:00:10`"?
6. What happens if your VPN/SSH disconnects, but your job has already been submitted to the CAAS cluster?

Self study and further reading

Install WinSCP or FileZilla and download/upload files into CAAS

Running a text editor (nedit or nano). Alternatively, you may edit a file from Windows, upload to CAAS, but will need to run: `dos2unix` to clean up the file.

Further help with Unix/Linux:

http://www.ee.surrey.ac.uk/Teaching/Unix/

Thank you for your time.

If you are facing issues accessing (or using) CAAS, your first point of contact ⇒ your tutor/lecturer

If your tutor/lecturer is not able to resolve your issues in accessing CAAS, then, please email:

servicedesk@monash.edu

On the subject, please include this tag: [FIT3143]